TECH ROUTE · REVERSE & REBUILD

拆解一款
自动化招聘 AI“智能体”

一款猎聘自动化桌面工具的完整制作逻辑

AI 拆解 JD · 接管真实浏览器 · 三层评分漏斗 · 批量沟通

从逆向分析到动手复刻，小白与技术人都能读懂

5 步流水线 | 20 个核心模块 | 3 层评分漏斗 | 144 行 LLM 适配器 | 0 自有服务端

simon · 一个不懂代码的 AI 极客

微信交流群 simonsxxs · github.com/simonsxxs · gitee.com/simonsxx

技术分享 · 仅供学习研究 · 基于安装包本地静态逆向（未安装、未运行目标程序），全部结论可溯源至源码 | 内容已脱敏：隐去原作者个人信息与内部基础设施地址

目录 CONTENTS

第一章　背景与概述：这是一个什么产品 4

1.1 一句话定位 4

1.2 五步流水线总览 4

1.3 为什么值得拆：五个值得借鉴的设计 5

1.4 本教程怎么读 6

第二章　技术栈与核心工具 7

2.1 技术栈总览：克制到只有 4 个依赖 7

2.2 核心工具与官方链接 7

2.3 三个反常识的选型决策 8

第三章　整体架构：一张图看懂 20 个模块 10

3.1 四层架构总览 10

3.2 模块依赖关系图 10

3.3 进程通信：40 个方法的安全桥 11

3.4 代码保护：入口字节码化 12

3.5 界面与交互设计：向导式五步 + 中式纸墨美学 13

第四章　正向链路：五步流程逐一拆解 15

4.1 JD 分析：把自然语言变成筛选规则 15

4.2 连接浏览器：接管，而不代替 16

4.3 提取简历：不解析 DOM，直调接口 18

4.4 深度分析：三层评分漏斗（产品灵魂） 20

4.5 批量沟通：只动 DOM，不碰接口 23

4.6 企业端镜像：更严风控下的另一套打法 24

第五章　逆向链路：异常、风控与自愈设计 26

5.1 四层风控护栏 26

5.2 操作节奏与点击频率全表（反风控核心参数） 26

5.3 浏览器不可用时：完整降级链 27

5.4 LLM 调用的重试与自修复 28

5.5 页面操作自愈与任务取消 28

第六章　瓶颈、风险与合规红线 29

第七章　动手复刻指南 30

7.1 复刻总路线：先 MVP，再完整版 30
7.2 逐模块复刻清单 31
7.3 需要自建的部分 31
7.4 复刻验收检查清单 32
附录 A　完整系统提示词原文（源码扒出） 33
参考链接 38
关于作者 39

CHAPTER 01 · 背景与概述

这是一个什么产品

本章回答三个问题：这个产品是什么、它的五步流水线长什么样、为什么它的做法值得拆开学习。读完本章，即使没有任何编程基础，也能看懂后面每一章在讲什么。

1.1 一句话定位

某自动化招聘智能体（化名，下文称「该产品」）是一款跑在 Windows 桌面上的「招聘自动化 + AI 筛选」工具：猎头或 HR 把岗位描述粘贴进去，AI 先把它拆成一份结构化的「候选人画像」；工具随后接管你电脑上**真实的浏览器**（你已经登录好猎聘账号），在猎聘搜索结果里批量提取简历、自动打分分级，最后自动勾选高分候选人并逐个打招呼。

它不是一个网站，也不是一个脚本，而是一个有完整界面的桌面软件——用 Electron（一个把网页打包成桌面应用的框架，Chromium 浏览器内核 + Node.js 运行时）制作，打包成 Windows 安装包分发，带激活码收费体系。本次拆解基于对安装包的**本地静态逆向**（只解包读代码，未安装、未运行），所有结论都能对应到源码文件。

> **一个诚实的命名观察：它其实是工作流（Workflow），不是智能体（Agent）**
>
> 严格地说，这个产品没有智能体的核心特征——没有「感知 → 规划 → 行动 → 反思」的自主循环，每一步做什么、调什么工具都是写死的线性流水线，大模型只出现在两个固定环节（拆 JD、最终评审）。叫它「智能体」是营销话术；工程上它是一套 **AI 增强的确定性工作流**。这不是贬义——恰恰是「决策交给规则、语义交给 AI、流程交给流水线」的分工，让它比真智能体更稳定、更可解释。读这份教程时，把它当工作流产品来学，收获更大。

表 1–1 产品基本面

属性	内容	说明
目标平台	猎聘网（liepin.com）	猎头端 h.liepin.com + 企业端 lpt.liepin.com 双端支持
产品形态	Windows 桌面应用	Electron 打包，NSIS 安装包（一种常见的 Windows 安装包格式）
分析版本	v1.0.16（稳定版）	内部工程名与作者信息均已脱敏
AI 能力来源	用户自备大模型 API Key	官方教程引导用户注册 DeepSeek，工具不倒卖额度
自有服务端	几乎没有	仅激活校验与自动更新两个轻量端点，业务全部本地完成

1.2 五步流水线总览

整个产品的使用流程只有五步，界面就是五个面板，一步一个。先把全貌记住，第四章会逐一拆开讲实现。



1. **JD 分析 —— AI 当「翻译官」**
   把一段自然语言岗位描述（JD，Job Description），翻译成机器能执行的结构化筛选条件：硬性要求、关键词维度、院校/年龄/年限等规则、推荐搜索词。

2. **连接浏览器 —— 接管你的浏览器，但不替你登录**
   工具通过浏览器调试接口（CDP，Chrome DevTools Protocol）接管本机浏览器。登录、搜索这两个最容易触发平台风控的动作，**始终由人手动完成**——这是全产品最重要的设计铁律。

3. **提取简历 —— 不爬页面，直接调数据接口**
   不解析网页元素（DOM，文档对象模型），而是在猎聘页面内部直接调用它的搜索数据接口，每页 30 人、最多 3000 人，拿到结构化简历数据后立即做第一轮关键词初筛，给出 S/A/B/C/D 等级。

4. **深度分析 —— 三层漏斗精筛**
   对初筛选中的人，用接口逐份调取完整简历（不开详情页）：先过六类结构化硬规则（院校/年龄/学历/年限/跳槽/证书），再让大模型扮演「资深猎头评审官」二次判断，输出精确分级和评语。

5. **批量沟通 —— 模拟人手点击**
   自动勾选 S/A 级候选人卡片，逐个点击「立即沟通」、选择职位、发出招呼。全程只做界面点击这类低风险动作。

给小白读者：什么是「接管浏览器」？

Chrome 系浏览器有一个官方调试功能：启动时加一个参数，就会在本机开一个「遥控端口」（--remote-debugging-port）。别的程序连上这个端口，就能看这个浏览器开了哪些网页、在网页里执行 JavaScript、点击按钮。这款产品就是用这个官方能力接管你日常用的浏览器——**你的登录状态、你的搜索条件都原样保留**，工具只是在旁边「搭顺风车」。这就是为什么它不需要你把账号密码交给它。

1.3 为什么值得拆：五个值得借鉴的设计

拆完 28 个源文件后，有五个设计是超越「一个招聘工具」本身、可以迁移到任何「AI + 网页自动化」产品的：

① 自写浏览器控制层，兼容国产浏览器

主流自动化框架（Puppeteer / Playwright）在 360 系浏览器上会失灵。作者绕开框架，用 WebSocket 直连调试协议自己实现了一个极简控制层，并用「给元素打标记」的方案绕开兼容性问题。所有「接管用户真实浏览器」的产品都能抄这个思路（见 4.2）。

② 专属浏览器档案 + 手动操作铁律

工具用独立的浏览器用户目录（profile）保存登录态，扫码一次永久有效；并且绝不代替用户做登录和搜索这两个高风险动作。把风控暴露压到最低的「人在环上」设计（见 4.2、5.1）。

③ 在页面里直接调数据接口，比解析网页快一个量级

简历列表不是从网页元素里抠出来的，而是借页面的登录状态直接请求后端数据接口，拿到干净的结构化 JSON。企业端更进一步：被动记录页面自己发出的请求来「借」搜索条件（见 4.3、4.6）。

④ 三层评分漏斗：规则省钱，AI 管语义

关键词初筛（免费）→ 结构化规则硬校验（免费）→ 大模型精判（花钱）。AI 只用在刀刃上，结果可解释、成本可控。任何「用 AI 做筛选」的产品都适用（见 4.4）。

⑤ 144 行代码写完的生产级大模型调用层

双协议自动适配、限流分级退避、输出格式错误自动回喂重试——一个文件踩完了调用大模型的所有常见坑，可以整体搬走（见 5.4）。

1.4 本教程怎么读

小白路线 · 了解产品逻辑

每章开头都有通俗解释，专业术语首次出现时会在括号里标注英文全称。重点读：第一章全景 → 第四章每节的「这一步在做什么」→ 第六章风险 → 第七章复刻路线。代码块可以跳过。

技术路线 · 照着复刻

重点关注：第三章架构与模块划分 → 第四章每节的代码片段与踩坑记录 → 第五章异常处理 → 第七章逐模块清单与验收标准。文中所有「实测坑」都来自源码里作者自己留下的注释。

CHAPTER 02 · 技术栈与工具

克制到只有 4 个依赖

> 这款产品的技术选型极其克制：全部依赖只有 4 个，UI 不用任何前端框架，大模型调用不装任何官方 SDK。本章先给全景，再解释三个「反常识」的选型决策——它们恰恰是复刻时最值得带走的东西。

2.1 技术栈总览

表 2–1 技术栈分层与选型理由

层	选型	为什么这样选
桌面壳	Electron（Chromium + Node.js）	用网页技术做桌面界面，一套代码出 Windows 安装包；招聘工具的目标用户（猎头/HR）几乎都在 Windows 上
打包分发	electron-builder + NSIS	业界标准组合，直接产出 .exe 安装包
主进程保护	bytenode（把 JS 编译成 V8 字节码）	入口和激活模块编译成 .jsc 防逆向，业务模块保持明文方便迭代（见 3.4）
浏览器自动化	自写极简 CDP 客户端（基于 ws 库）	不用 Puppeteer 高层接口，为了兼容 360 系国产浏览器（见 4.2）
浏览器获取	@puppeteer/browsers + chrome-for-testing	用户没装合适浏览器时，自动下载便携版 Chrome 兜底
大模型调用	纯 fetch 手写适配器（144 行）	零 SDK 依赖，同时兼容 OpenAI 与 Anthropic 两种接口格式（见 5.4）
界面 UI	原生 HTML/CSS/JS 单页，3 个文件	五个面板的向导式界面不需要框架，少一层依赖少一层坑
进程通信	contextBridge + IPC 白名单	界面只能通过约 40 个登记过的方法调用主进程能力（见 3.3）
日志与更新	electron-log + electron-updater	打包后控制台不可见，日志文件是唯一诊断途径；更新包托管在对象存储上
收费激活	机器码 + 激活码/卡密	字节码保护，本地存激活状态（见 7.3，复刻需自建）

2.2 核心工具与官方链接

复刻时用到的全部开源项目与官方资源（文末「参考链接」章有汇总）：

表 2-2 涉及的开源项目与工具

工具 / 项目	用途	官方链接
Electron	跨平台桌面应用框架（网页技术做桌面软件）	electronjs.org
electron-builder	打包与 NSIS 安装包生成	github.com/electron-userland/electron-builder
bytenode	把 Node.js 代码编译成 V8 字节码（防逆向）	github.com/OsamaAbbas/bytenode
ws	Node.js 的 WebSocket 客户端，CDP 通道的底座	github.com/websockets/ws
CDP 官方文档	Chrome 开发者工具协议（浏览器遥控接口）说明书	chromedevtools.github.io/devtools-protocol
@puppeteer/browsers	浏览器版本探测与便携版下载	pptr.dev/browsers-api
chrome-for-testing	谷歌官方发布的「专供自动化」Chrome 便携版	googlechromelabs.github.io/chrome-for-testing
npmmirror 镜像	上述便携版的国内镜像下载源（双源兜底之一）	registry.npmmirror.com
electron-log	主进程日志落盘	github.com/megahertz/electron-log
DeepSeek 开放平台	官方教程推荐的大模型供应商（用户自备 Key）	platform.deepseek.com
OpenAI 兼容协议	大模型接口事实标准之一（/v1/chat/completions）	platform.openai.com/docs/api-reference
Anthropic 协议	另一种主流大模型接口格式（/v1/messages）	docs.anthropic.com

2.3 三个反常识的选型决策

决策一：不用 Puppeteer，自己写 170 行浏览器控制层

Puppeteer 是谷歌官方的浏览器自动化库，几乎是这类需求的默认答案。但作者实测发现：**360 Chrome X（大量猎头在用的国产浏览器）不支持 Puppeteer 依赖的 callFunctionOn 指令**。于是作者放弃框架，用 WebSocket 直连调试协议，只实现自己用到的十几个能力（连接、导航、执行 JS、找元素、等元素、读 Cookie），并用「给元素打 data-cdp-id 标记」的方案替代不兼容的指令（完整实现见 4.2.3）。

> **复刻启示**
>
> 当你的产品要接管的是「用户电脑上已经装好的、五花八门的真实浏览器」时，框架的兼容性承诺会失效。自写薄控制层 = 完全掌控行为 + 精准兼容目标环境，代码量并不大。

决策二：大模型调用不装 SDK，纯 fetch 手写

调用大模型只有两件事：按格式拼一个 HTTP 请求、把返回的文本取出来。官方 SDK 会引入一长串传递依赖。作者用 144 行原生 fetch 实现了双协议适配（OpenAI 格式 / Anthropic 格式自动判断）、鉴权头差异处理、超时与重试——复刻时这个文件可以几乎原样搬走（见 5.4）。

决策三：UI 不用框架，3 个文件写完整界面

界面就是「设置 + 五个步骤面板 + 结果表格」的向导式单页应用。作者用原生 HTML/CSS/JS 完成（最大的 renderer.js 约 65KB），没有 React/Vue。好处是零构建、零框架升级负担；代价是交互逻辑全靠手写——对「功能型工具」而非「复杂应用」来说，这是划算的取舍。

CHAPTER 03 · 整体架构

一张图看懂 20 个模块

> Electron 应用天生是「两个世界」：你看到的界面（渲染进程）和真正干活的幕后（主进程）。本章用一张分层图和一张依赖图讲清楚这个产品的模块怎么摆、话怎么说、门怎么锁。

3.1 四层架构总览

从上到下，数据和能力逐层流动。每一层只做自己的事，层与层之间只有窄窄的通道——这是这个产品「小而稳」的根本原因。

① 界面层（渲染进程） 原生 HTML/CSS/JS 单页 · 五个步骤面板 + 设置页 · 结果表格 + 星级徽章 · 无任何前端框架	用户看得见 摸得着的部分
② 安全桥（preload.js） contextBridge 白名单 · 约 40 个 ipcRenderer.invoke 方法 · 7 个进度推送事件通道 · 界面无法越权调用	唯一的门 白名单制
③ 业务层（主进程 core/，20 个模块） JD 拆解 · LLM 适配器 · 浏览器生命周期 · CDP 控制层 · 搜索接口直调 · 评分引擎 · 六类规则 · 院校库 · 批量沟通 · 企业端六件套 · 历史记录	产品的大脑 全部业务逻辑
④ 外部世界 用户本机浏览器（CDP 9222 / 9224 端口） · 猎聘数据接口 · 大模型 API（用户自备 Key） · 对象存储更新源 · 激活校验端点	没有自有 业务服务端

图 3–1　四层架构：界面 → 安全桥 → 业务核心 → 外部世界

3.2 模块依赖关系图

主进程入口只有 3 行代码，真正的业务在 `core/` 目录的 20 个模块里。下图是逆向分析后确认的模块依赖关系（实线为确认依赖）：

读图要点

注意三个「枢纽」：**llm.js** 被 JD 拆解和深度分析共用（全产品只有这两个环节调用大模型）；**score.js** 被提取初筛和深度分析共用（同一套评分逻辑，两档严格程度）；**searchApi.js** 同时服务猎头端和企业端（一套接口直调代码，两端复用）。枢纽模块少而精，是代码量控制住的关键。

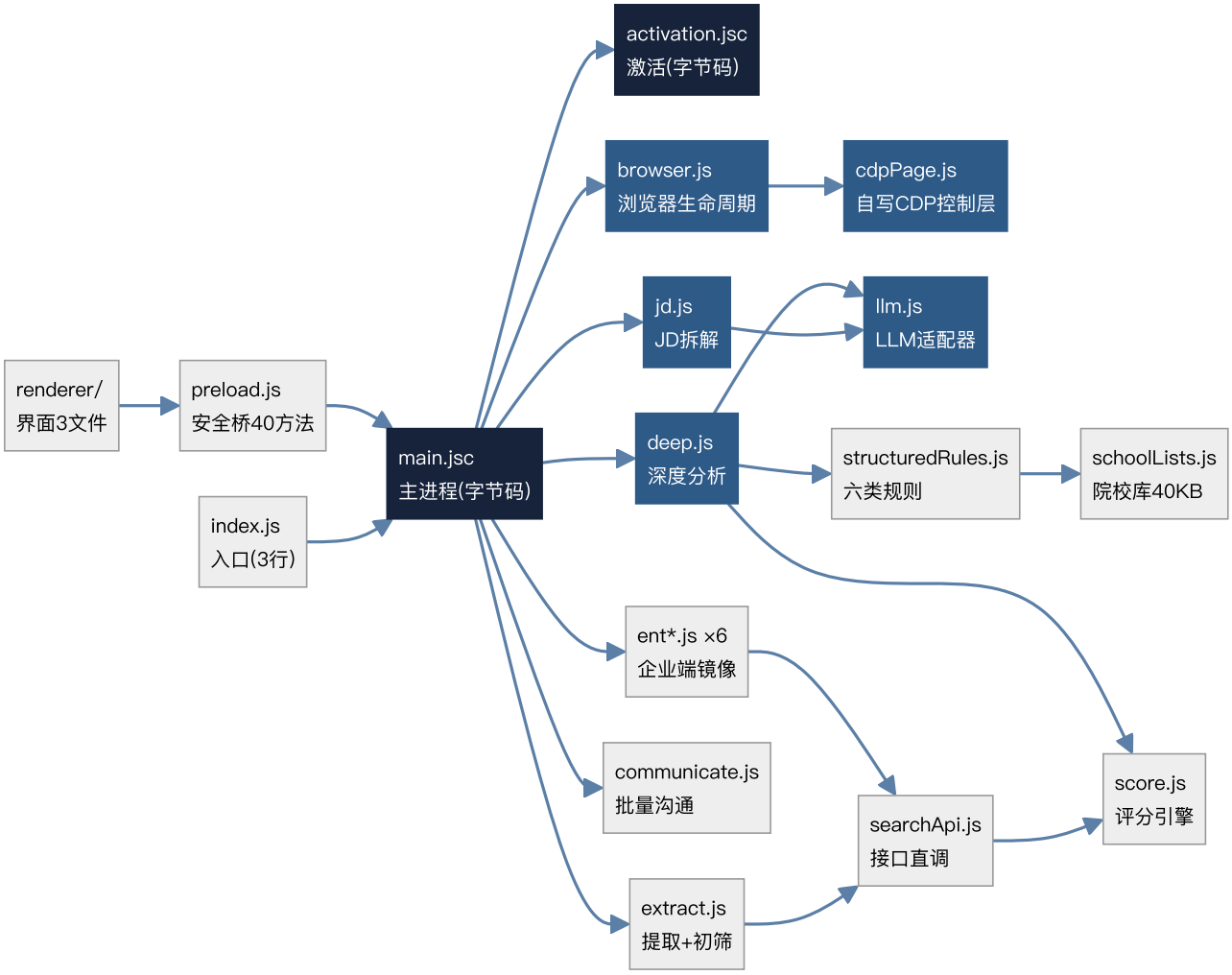


图 3-2　core/ 模块依赖关系（逆向分析确认）

3.3 进程通信：40 个方法的安全桥

Electron 的渲染进程（界面）默认不能碰文件、不能开网络以外的系统能力——所有「超能力」都在主进程。两者之间靠 IPC（Inter-Process Communication，进程间通信）传话。这个产品的做法是教科书级的：**preload.js 用 contextBridge 只暴露一份写死的白名单**，界面想做什么，只能点菜单上有的菜。

◆ preload.js 节选 —— 安全桥的方法登记（约 40 个，按业务分组）

```
contextBridge.exposeInMainWorld('api', {
  // JD 分析
  jdAnalyze:         (jdText) => ipcRenderer.invoke('jd-analyze', jdText),
  // 浏览器
  browserLaunch:     (dir) => ipcRenderer.invoke('browser-launch', dir),
  browserConnect:    () => ipcRenderer.invoke('browser-connect'),
  // 工作流（提取 / 深析 / 沟通，均带 cancel）
  extractStart:      (jd) => ipcRenderer.invoke('extract-start', jd),
  deepStart:         (p) => ipcRenderer.invoke('deep-start', p),
  deepCancel:        () => ipcRenderer.invoke('deep-cancel'),
  communicateStart: (o) => ipcRenderer.invoke('communicate-start', o),
  // 企业端全套镜像（配额/提取/深析/沟通/PDF下载）
  entQuota:          () => ipcRenderer.invoke('ent-quota'),
  entDownloadStart: (p) => ipcRenderer.invoke('ent-download-start', p),
  // 进度推送（主进程 → 界面，7 个事件通道）
  onExtractProgress: (cb) => ipcRenderer.on('extract-progress', (e,d) => cb(d)),
  onDeepProgress:    (cb) => ipcRenderer.on('deep-progress',    (e,d) => cb(d)),
  onCommLog:         (cb) => ipcRenderer.on('comm-log',         (e,d) => cb(d)),
  // ……更新、激活、历史记录等，共约 40 个方法
});
```

复刻要点

白名单桥接的三个好处：①安全——界面被注入恶意代码也拿不到菜单外的能力；②可维护——主进程内部随便重构，只要 40 个方法签名不变，界面就不用动；③进度体验——长任务（提取 3000 人、深析几百人）通过事件通道实时推送进度条，并且每个长任务都配了 cancel 方法，用户随时可以叫停。

3.4 代码保护：入口字节码化

Electron 应用的源码默认打包在 app.asar 里，任何人都能解开看。这款产品的策略是「**锁门但不锁窗**」：业务代码明文（方便自己快速迭代），但入口和激活模块用 bytenode 编译成 V8 字节码（.jsc），逆向者只能看到一堆二进制：

◆ index.js 全文 —— 入口只有 3 行，真正的逻辑在编译后的 main.jsc 里

```
'use strict';
require('bytenode');          // 注册 .jsc 字节码加载器
require('./main.jsc');        // 主进程逻辑：已是 V8 字节码，无法直接阅读
```

给小白读者：什么是字节码保护？

JavaScript 是明文文本，谁都能读。bytenode 把它「预编译」成 V8 引擎直接执行的字节码（类似把文章转成只有特定机器能读的编码）。不是绝对安全（可以被 dump 分析字符串），但把「读懂代码」的成本从记事本级别拉高到了专业逆向级别。本教程的分析中，激活模块的具体协议就因此无法完全还原（见 6.1）。

3.5 界面与交互设计：向导式五步 + 中式纸墨美学

界面层只有一个 HTML + 一个 CSS + 一个 JS（约 1300 行），但交互完整度远超一般小工具。拆开看有三层结构：

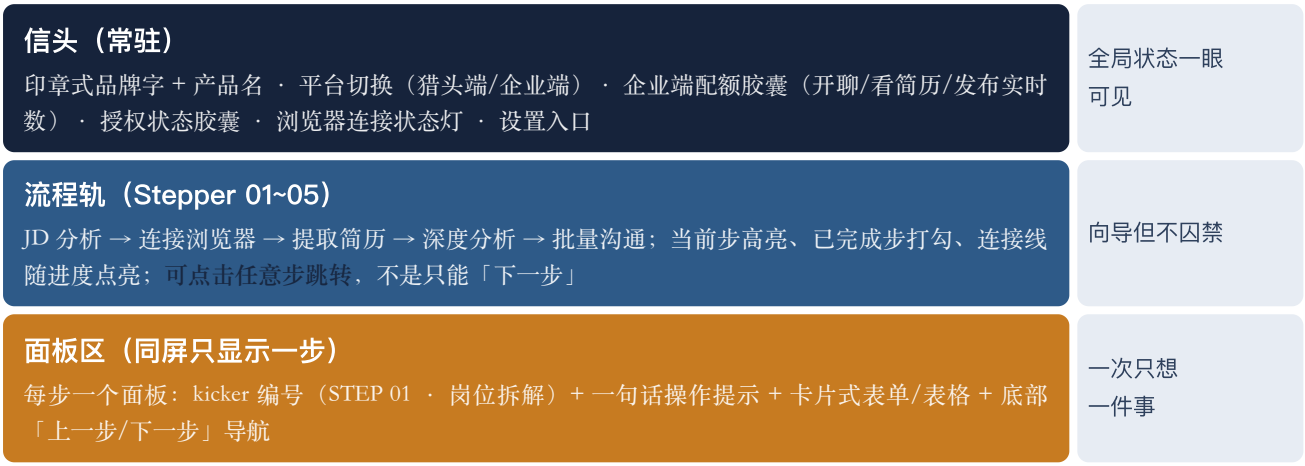


图 3-3　界面三段式结构：信头 / 流程轨 / 面板区

3.5.1 视觉设计：暖信纸 + 朱砂印章

配色不是常见的科技蓝，而是一套「中式文书」美学——和「简历、文书、猎头」的产品气质是呼应的：

表 3-1 UI 设计变量（styles.css · :root）

变量	色值	用途
`--paper`	#f6f5f1 暖信纸底	全局背景
`--ink`	#201e1a 墨色	正文文字
`--seal`	#b3372c 朱砂红	品牌印章、kicker 编号、危险操作按钮
`--hairline`	#e5e3dc 暖发丝线	卡片描边与分隔
`--radius`	14px 大圆角	卡片与弹窗；按钮 9px；徽章 999px 胶囊
字体分工	宋体系衬线 / 等宽 / 系统黑体	品牌字与 kicker 用衬线（Songti SC）；编号数字用等宽（SF Mono）；正文用系统黑体

另有一个安全细节：页面头里写了 CSP（内容安全策略，Content Security Policy）`default-src 'self'`——界面只允许加载应用自身的资源，从根上断了注入外部脚本的路。

3.5.2 交互逻辑：七个值得抄的细节

1. **AI 输出一律可人工修改**
   JD 拆解结果不是只读展示，而是填进可编辑表单（标题/学历/年限/层级/薪酬/硬性条件/维度关键词都能改）——AI 起草、人定稿，纠正成本为零

2. **批量选择有场景化快捷键**
   提取页有「全选 / 选 B 级以上 / 清空」；深析页有「全选 / 选 S/A 级 / 清空」——每个场景的默认动作都替你预演过

3. **长任务三件套：进度条 + 计时 + 预估剩余**
   深析进度条实时刷新，带「⏱ 已用 02:31 · 预估 05:40」式倒计时（按已完成速度估算），等待不再焦虑

4. **暂停按钮的文案即承诺**
   批量沟通的停止按钮写作「暂停（下次断点续传）」——按钮文案直接告诉你数据不会丢，因为 doneRids 确实实时持久化了（见 4.5.2）

5. **结果可回看、可复用、可导出**
   历史列表每条带 S~D 分级分布徽章；点候选人姓名弹窗看完整简历；一键导出 CSV；二次匹配不用再调接口

6. **平台切换是「换芯」而非「换皮」**
   切到企业端：配额胶囊出现、「选 B 级以上」隐藏（企业端提取不评分、深析才评分）、「批量下载简历 PDF」按钮出现——界面按平台能力精确增减

7. **设置页「先测试再保存」**
   API Key 旁有「测试连接」，浏览器路径旁有「自动检测」——配置类错误在保存前就暴露，而不是等跑任务时才炸

商业化界面也内置在授权中心里

未激活时授权弹窗是阻断式的（盖住全屏、不可关闭）；已激活后变成顶栏可随时点开的「查看/续期」面板。弹窗里集成了完整销售闭环：复制机器码 → 联系管理员购买（联系方式已脱敏）→ 激活码/卡密激活 → 邀请码裂变（邀请好友购买双方各得 30 天）。定价分四档：月度 ¥99 / 季度 ¥249 / 年度 ¥799 / 永久 ¥1999——个人开发者工具的典型定价结构，复刻时可参考。

CHAPTER 04 · 正向链路

五步流程逐一拆解

> 本章是全教程的主体：五步流程，每步都讲清楚「在做什么 → 怎么实现 → 踩过什么坑 → 复刻怎么抄」。五步之间靠一份「JD 画像 JSON」贯穿：第一步产出的画像，是后面每一步的输入。

4.1 JD 分析：把自然语言变成筛选规则

这一步在做什么：用户粘贴一段岗位描述，大模型（LLM，Large Language Model）扮演「资深猎头顾问」，输出一份严格格式化的 JSON——这就是后续所有筛选的「判决书模板」。

4.1.1 画像的字段设计

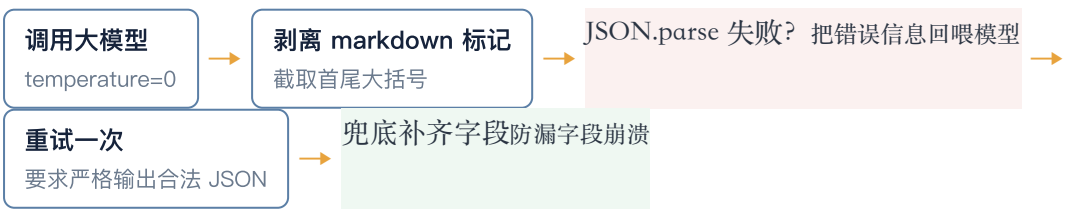
系统提示词（System Prompt，给 AI 的角色与任务说明书）完整固化在源码里，约 45 行。它要求模型输出的字段设计非常讲究，直接决定了后面漏斗的形状：

表 4–1 JD 画像 JSON 的字段设计（jd.js · SYSTEM_PROMPT）

字段	作用	设计心机
`hardRequirements`	硬性技术词，缺一即 D 级淘汰	强制「原子词」，禁止把多词拼成一句（防整串匹配永远不中）；宁少勿滥
`mustHaveExp`	必须具备的领域经验（语义门槛）	与技术词互补：简历不会字面写出「跨国管理经验」，要靠 AI 语义判断
`structuredRules`	范围型硬条件：院校/年龄/学历/年限/跳槽/证书	简历文本里根本不会出现「985」这种判断，必须结构化判定（见 4.4.2）
`dimensions`	三个关键词维度：核心技能/领域经验/素质背景	每维度 8~20 个「能在简历里字面命中」的具体词，禁用「相关」「的」这类泛词
`salaryRange`	薪酬区间（统一折算成月薪数字）	13 薪按年薪÷13 折算；给后面薪酬规则引擎用（见 4.4.4）
`searchKeys`	推荐搜索词	指导用户在猎聘上手动搜索时用什么关键词

4.1.2 输出坏了怎么办：JSON 自修复

大模型偶尔会输出非法 JSON（多了逗号、引号没转义）。这个产品的做法不是报错，而是**把解析错误原样回喂给模型，让它自己改一遍**——一次自修复机会，实测能解决绝大多数偶发格式问题：



实测坑 · 模型脏数据的归一化

即使 JSON 合法，字段也可能是脏的。源码里有一段细致的兜底：规则类型不在六类白名单内直接丢弃；数值字段强制转 Number——特别注明 `空字符串会被 Number('') 归一成 0`，如果不排除，一条「年龄最大 0 岁」的规则会把所有候选人一票否决。复刻时这类「防御性归一化」比提示词本身更能决定系统稳定性。

4.2 连接浏览器：接管，而不代替

这一步在做什么：工具找到（或帮你装好）一个浏览器，用调试端口启动它，然后「挂上遥控器」。登录猎聘、搜索岗位这两件事由用户自己在这个浏览器里完成——工具只读结果。

4.2.1 五级浏览器检测级联

用户电脑里有什么浏览器是不可控的。产品的解法是按优先级逐个探测，全没有就自动下载一个便携版：

1. **360 Chrome X**
   老用户首选，专属 profile 里可能还存着以前的登录 Cookie，免扫码
2. **Google Chrome（版本 ≥ 110）**
   读安装目录下的版本号文件夹判断主版本，过旧跳过
3. **Microsoft Edge（版本 ≥ 130）**
   老版本 Edge 跨站跳转时调试会话会断（实测坑），低于下限直接跳过
4. **360 安全浏览器 → 360 极速浏览器**
   继续兜底，覆盖国产浏览器用户群
5. **全都没有 → 自动下载便携版 Chrome**
   chrome-for-testing 官方自动化专用版，谷歌官方 + npmmirror 国内镜像双源下载，PowerShell / tar 双方案解压

4.2.2 启动参数里的门道

◆ browser.js · launch() —— 浏览器启动参数（猎头端）

```
const args = [
  `--remote-debugging-port=9222`,            // 开遥控端口（猎头端 9222 / 企业端 9224）
  '--start-maximized',
  '--disable-blink-features=AutomationControlled',  // 反检测：抹掉「正被自动化」的浏览器指纹
  '--disable-automation',
  `--user-data-dir=${prof}`,                 // 专属用户目录：登录态持久化的关键
  '--no-first-run', '--no-default-browser-check',
];
```

给小白读者：专属用户目录（user-data-dir）为什么重要

浏览器的登录状态（Cookie）存在「用户目录」里。工具每次都用同一个专属目录启动浏览器，于是**扫码登录一次，以后永远免登**。同时这个目录和你日常上网的目录隔离，互不影响。这是「工具复用人工登录态」设计的地基。

4.2.3 自写 CDP 控制层：data-cdp-id 方案

这是全产品技术含量最高的一段。CDP（Chrome DevTools Protocol，浏览器调试协议）本质是「用 WebSocket 给浏览器发 JSON 指令」。作者用 ws 库直连，自己实现了两个类：`CDPConnection`（管连接和指令收发）和 `CDPPage`（管单个标签页）。

最巧的设计是元素定位：标准做法 callFunctionOn 在 360 Chrome X 上不支持，作者改用「**给目标元素打一个临时标记属性，再用 CSS 选择器找它**」：

◆ cdpPage.js 节选 —— 用 data-cdp-id 标记替代 callFunctionOn

```
async $(selector) {
  const cdpId = `cdp-${++_elCounter}`;
  // 在页面里找到元素，给它打一个临时标记属性
  const r = await this._conn.send('Runtime.evaluate', {
    expression: `(() => { const el = document.querySelector(${JSON.stringify(selector)});
      if(el){ el.setAttribute('data-cdp-id','${cdpId}'); return true; } return false; })()`,
    returnByValue: true,
  }, this._sessionId);
  if (!r?.result?.value) return null;
  return { __elementHandle: true, _cdpId: cdpId };    // 句柄只是一个标记名
}
// 之后要操作这个元素时，用 document.querySelector('[data-cdp-id="cdp-7"]') 找回来；
// 用完立即 removeAttribute 清理，不污染页面 DOM。
```

这个方案的额外红利

所有点击操作统一走 `page.evaluate(el => el.click())`（在页面上下文里直接调元素的 click 方法），而不是框架的 ElementHandle.click()——后者会先滚动到元素位置（scrollIntoView），在猎聘的虚拟列表里会卡死。这个细节在批量沟通章节（4.5）是头号铁律。

4.2.4 双平台隔离

表 4-2 猎头端与企业端的隔离设计

	猎头端（h.liepin.com）	企业端（lpt.liepin.com）
调试端口	9222	9224（独立实例，互不干扰）
浏览器	五级检测级联，有啥用啥	**固定 Google Chrome**（企业端风控更严）
用户目录	360-cdp-profile / liepin-cdp-profile	enterprise-cdp-profile（独立档案）
数据获取	页面内直调搜索接口（主动）	钩子被动记录页面自身请求（见 4.6）

4.3 提取简历：不解析 DOM，直调接口

这一步在做什么：用户已在猎聘搜索页上看到候选人列表。工具不一个个去抠网页上的卡片文字，而是在页面内部直接调用猎聘自己的搜索数据接口，拿到干净的 JSON 数据，顺手做第一轮初筛打分。

4.3.1 核心手法：页面上下文 fetch

因为代码是在猎聘页面里执行的，浏览器会自动带上登录 Cookie 和防伪令牌（XSRF-TOKEN，跨站请求伪造防护令牌），工具只需要手动补几个签名头：

请求（在猎聘页面上下文内发出）

```
POST https://api-h.liepin.com/api/com.liepin.searchfront4r.h.search-resumes
Content-Type: application/x-www-form-urlencoded
X-XSRF-TOKEN: <从 document.cookie 自动读取>
X-Fscp-Std-Info: {"client_id": <从 window.__FE_CLIENT_ID 运行时读取>}
X-Fscp-Trace-Id: <每次生成的 UUID>

body: searchParamsInputVo={职位code,关键词,城市,学历,薪资…}&logForm={skId,searchScene}
```

响应（结构化 JSON，节选）

```
{
  "flag": 1,
  "data": {
    "totalCnt": 41,
    "resList": [ { "resName": "…", "resEdulevelName": "硕士",
                   "skillTags": ["…"], "workExpFormList": ["…"] }, … ],
    "skId": "翻页凭证，下一页必须回传"
  }
}
```

4.3.2 动手前先等一等：搜索稳定性门禁

这是源码里一段很容易被忽略、但极重要的操作逻辑（2026-09-05 强化）。猎聘的搜索是渐进式的：刚搜完时，接口返回的总人数会从宽到窄收敛（真机实测：某关键词+职位组合从 294 人收敛到 181 人，约需 40~80 秒）。如果搜完立刻提取，会把「中间态总数」锁死，提取结果和页面显示对不上。

产品的解法是**提取前先过一道门禁**：读页面上的「共 X 位人选」（这个值一开始就是对的）作为收敛目标，再读接口总数，两者差距 >3 就每 5 秒重读一次接口，直到对齐或等满 60 秒。正常情况（搜索已完成很久）第一次读就对齐，零额外等待。门禁顺带拿到第 1 页数据，主循环直接从第 2 页开翻——少一次请求，也就少一分风控触发面。



4.3.3 三个用真机踩出来的坑

实测坑 · pageSize 只能传 30

后端翻页偏移缓存按 30 条粒度存储。传 100 只能拿到第 1 页，第 2 页起报「缓存中不包含偏移量信息」——重试、回传 skId 都无效，唯 30 能翻页（源码注释：2026-08-30 真机实测）。猎聘搜索展示上限约 3000 人，即最多翻 100 页。

实测坑 · 签名头要跟着平台前端升级

2026-08-31 猎聘前端升级后，client_id 不再是固定值，要从页面运行时变量 window.__FE_CLIENT_ID 动态读，还要补 X-Fscp-Fe-Version 头。**凡依赖平台接口细节的产品，都要预留「持续对抗升级」的维护预算**（见 6.1）。

实测坑 · 搜索条件要全量透传，丢一个字段结果就对不上

工具从页面的 sessionStorage（浏览器会话存储）里读出用户手动搜索的全部条件再转发。源码记录：只传第一个职位 code 时，3 职位的搜索结果从 41 人缩水到 22 人；丢掉「近 2 段工作均不低于 2 年」字段时，419 人对不上页面的 259 人。保证「**工具提取结果 = 用户页面所见**」是这类产品的信任根基。

4.3.4 顺手完成初筛：语义分级而不是算分

拿到每人的卡片数据后，立即用关键词命中做初筛。评分引擎（score.js，约 100 行）的思路是「**不算分，直接定级**」：

表 4–3 五级语义分级规则（score.js · gradeCandidate）

等级	含义	判定条件
S ★★★★★	完全符合 · 背景优秀	硬性条件全过 + 核心技能命中 ⩾3 词 + 三维度全覆盖 + 名校/硕博
A ★★★★☆	符合要求	硬性条件全过 +（核心技能命中 ⩾3 词 或 三维度全覆盖）
B ★★★☆☆	基本符合	核心技能命中 ⩾2 词 或 至少覆盖 2 个维度
C ★★☆☆☆	沾边	硬性条件过了，但几乎没有实质技能命中
D ☆☆☆☆☆	不匹配	硬性条件缺失（深析阶段一票否决）

关键分寸：初筛阶段硬性词「只记录，不否决」

同一份评分代码有两档严格度：列表卡片的文本很稀疏（只有技能标签、职位、公司），简历里真有的词卡片上不一定显示——**卡片上缺词 ≠ 真缺**。所以初筛时硬性条件缺失只记下来（hardMissing），不判 D；到了深度分析阶段拿到完整简历，才启用一票否决。这个「两段严格度」设计避免了大量误杀。

4.4 深度分析：三层评分漏斗（产品灵魂）

这一步在做什么：对初筛后勾选的候选人，逐份调取完整简历（注意：**不开详情页**，而是直接调简历数据接口，原因见下），然后过三道关卡，每层比上一层更贵、也更准。这是整个产品最值得抄的设计——**把「哪些判断交给规则、哪些交给 AI」划得清清楚楚。**

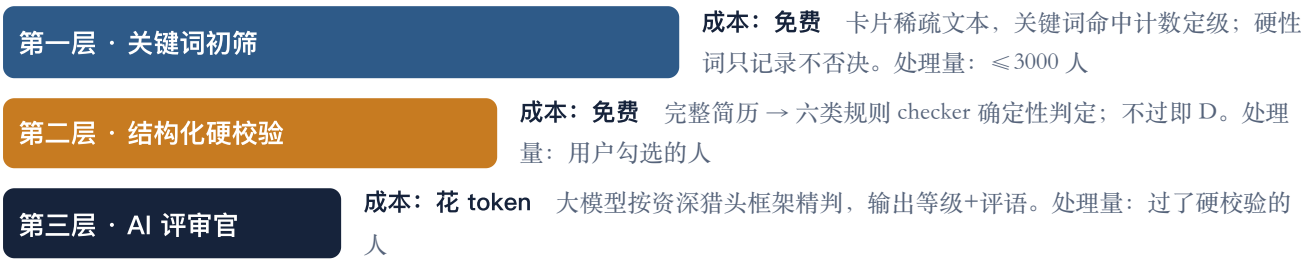


图 4–1　三层评分漏斗：越往下越贵、越准，处理量越小

4.4.1 简历怎么取来：resume-view 接口直取（B 方案）

早期版本是「逐个打开候选人详情页，从 DOM 里抠简历全文」。2026-08-24 实测定性后发现这条路的致命伤：**每开一个新标签页都会争抢会话（session），导致搜索页被冲成空白页**。于是换成 B 方案——在搜索页上下文里直接 fetch 猎聘的简历详情接口 `resume-view`（纯读单份简历、不带翻页凭证），实测连续请求 40 秒搜索页不空白。

接口返回的 `resumeDetailVo` 是结构化 JSON，字段与详情页 DOM 一一对应：

表 4–5 简历接口字段映射（mapResumeDetail）

接口字段	含义	解析后的用途
`workExperiences[].rwDuty`	每段工作的职责业绩	拼进全文做关键词匹配；截取前 150 字喂给 AI 评审
`workExperiences[].rwCompname / rwTitle`	公司名 / 职位名	跳槽段数合并判定（去括号归一）；工作摘要
`startYear / rwStart` 等	起止时间（兼容数字与 YYYYMM 两种格式）	经验年限、经验时效（是否近期）判定
`eduExperiences[]`	教育经历（redSchool/redSpecial/redDegreeName + schoolGradeNames 院校标签 + redTz 统招标记）	第一学历 985/211 判定（标签优先、本地名单兜底）
`usercTags`	技能标签	关键词匹配（最多取 20 个喂 AI）
`jobWant.wantSalaryShow`	期望薪资	薪酬规则引擎输入（解析成月薪数字区间）

`selfDescr` / `projectExperiences` / `languages`	自我评价 / 项目 / 语言	拼进全文，供关键词与 AI 综合判断

提取节奏是「**5 路并发 + 批次间停顿 300 毫秒**」：每批 5 份简历同时 fetch，批与批之间歇 300ms 避免连续请求触发接口限流；接口返回空（工作/教育都缺）时 500ms 后重试一次，仍空则标记「提取失败，结果不可信」而不是误判淘汰。取完简历后，AI 精判环节再开 **10 路并发**调大模型——读接口和调 AI 分开限流，互不拖累。

4.4.2 第二层：六类结构化规则 checker

关键词匹配做不了「985/211」「35 岁以内」「跳槽不超过 3 段」这类**范围型**判断——简历文本里根本不会字面出现这些结论。产品在 JD 分析阶段就把它们转成结构化规则，深析时先把简历抽成「事实」（院校、年龄、学历、年限、工作段数、证书），再逐条判定：

表 4–4 六类结构化规则（structuredRules.js）

规则类型	规则示例（JSON）	判定要点
school 院校	`{type:'school', require:['985','qs100'], list:[…]}`	本地院校库精确匹配 + 简历自带标签双通道；QS 按档位精确判定
age 年龄	`{type:'age', max:35}`	兼容「30」「30岁」「1990年生」多种写法
degree 学历	`{type:'degree', min:'硕士'}`	学历等级制：博士7 > 硕士/MBA 6 > 本科5 > 大专4…
years 年限	`{type:'years', min:3, max:10}`	解析「工作4年」「8-10年」等写法
jobHop 跳槽	`{type:'jobHop', withinYears:5, minJobs:3}`	「5年3跳淘汰」；同公司不同项目去括号合并，防误判
cert 证书	`{type:'cert', list:['PMP']}`	全文须含其一

院校匹配的防误判细节（40KB 院校库的用法）

①两字简称只做精确匹配——「南大」不会误命中「济南大学」；②独立学院排除——「厦门大学嘉庚学院」不算「厦门大学」（前缀延展即排除）；③QS 排名按 JD 原文档位判定（QS 前 50 就查 qs50 档，不一律按 300）。底库覆盖 985 / 211 / 双一流补充 / QS 排名榜。最狠的一条：**无法校验时按不通过处理**——宁可保守漏杀，也不让范围外的人蒙混到 S/A。

4.4.3 第三层：AI 评审官的提示词工程

过了硬校验的候选人，连同 JD 画像一起交给大模型。JUDGE_SYSTEM 提示词约 90 行，是一个「资深猎头」的完整判断体系，结构可以直接搬进任何 AI 评审场景：

框 **三步判断框架**

①理解岗位本质（要解决什么问题，不是 JD 写了什么）→ ②评估候选人（实质匹配而非字面匹配）→ ③识别常见误判（六条反机械匹配铁律）

铁 **四条最高优先级铁律**

院校事实以本地名单为准（禁止模型凭记忆重判）；hardMissing 非空必须判 D；必备经验完全缺失必须判 D；相关经验累计 ≤2 年且非近期不得高于 C

出 **结构化输出**

等级（S~D）+ 强项 + 风险 + 差距 + 致命缺口 + 一句话结论（结论必须与等级严格一致）

尺 **总原则：宁严勿松**

「打分必须客观，有实质差距就如实降级，禁止用『有疑问给高一级』来掩盖硬伤」——筛选类 AI 的口径基准

为什么「院校事实以本地名单为准」是神来之笔

大模型对「某大学是不是 985」这类事实的记忆并不可靠，会一本正经地胡说。产品先用本地院校库把事实算好，再以字段形式喂给模型，并明令禁止模型推翻。这是「**确定性计算管事实，大模型管语义**」分工原则的教科书示范——任何用 AI 做判断的系统都应该这样划分事实权。

4.4.4 薪酬匹配：独立的规则引擎

薪酬判断不交给 AI，而是一个小规则引擎：候选人期望下限 ≤ 岗位上限即预算内；超 30% 内提示「略超」；超 2 倍直接判「薪酬不现实」。防止「低薪岗配高薪人选」被 AI 误判成 S 级——又一个「能算清楚的就不麻烦 AI」的实例。

4.4.5 三层结果怎么合并：定级优先级

每个候选人最终只有一个等级，三层结果的合并顺序是写死的，优先级从高到低：

硬校验不过直接 D → fatalGaps 非空直接 D → **AI 建议有效** 采用 S~D → AI 失败规则兜底

也就是说：**AI 只能往下压，不能往上抬**——硬性条件不过的人，AI 说破天也是 D；而 AI 失败时系统不会崩溃，退回确定性规则等级，并在结果里标注「Claude 精判失败，已用规则兜底」。每个人最终展示的「关键匹配点」由维度命中、院校、层级判断、强项/短板/差距、一句话结论拼装而成，淘汰的人必附淘汰原因。

4.4.6 结果存储与复用：本地存档的克制设计

深度分析结果存在应用数据目录的 `deep_history.json` 里（纯本地文件，不上传任何服务器），设计上有三个细节值得学：

- **只留最近 30 条**，防止文件无限膨胀；

- **长文本截断**：每段工作职责截到 300 字、自我评价截到 500 字；
- **简历全文（fullText）不落盘**——加载历史时从结构化字段（工作/教育/技能/期望/自评）重新拼接出来。既省空间，又减少敏感简历原文的本地暴露面。

存档支持回载与删除，界面可导出 CSV；回载的历史还能做「二次深度分析」——用户补充几个关注要点关键词，在已存全文上重新匹配，不用再调接口、不用花 token。

4.5 批量沟通：只动 DOM，不碰接口

这一步在做什么：回到搜索结果页，自动勾选目标候选人卡片，逐个点击「立即沟通」、在弹窗里选择职位、发出招呼。与提取阶段相反——**这里完全不用接口，只用界面点击**，因为打招呼是平台最敏感的写操作，模拟人的点击路径最不容易触发风控。



源码开头内建了五条铁律，每一条都是踩坑换来的：

4.5.1 操作路径拆解：对一个人打招呼的完整动作序列

以下是 `communicateOne()` 对单个候选人的真实操作序列（全部在页面上下文里用 evaluate 执行，括号内是源码里的等待节奏）：

1. **定位卡片**
   遍历简历卡片，用隐藏复选框 input[name="res_id_encode"] 的值匹配目标 resumeId，找到卡片里的按钮
2. **读按钮文案分支**
   「继续沟通」→ 跳过（跳 IM 会丢会话）；「立即沟通」→ 点击；找不到 → 记录失败
3. **轮询等弹窗（400ms × 最多 8 次）**
   最快 0.4 秒、最慢 3.2 秒检测到沟通弹窗，比固定等 3 秒快得多
4. **打开职位下拉（mousedown → mouseup → click 三连）**
   Ant Design 的 select 组件只认 mousedown 开下拉，直接 click 无效（等 800ms）
5. **虚拟列表里找目标职位（每轮 150ms，最多 40 轮）**
   下拉是虚拟列表，必须小步长 30px 滚动 + 手动派发 scroll 事件逼它刷新；职位名被省略号截断也能「截断感知」匹配；滚到底没找到提前退出
6. **兜底：找不到职位就「不选择职位开聊」**
   点兜底按钮（等 1500ms），然后轮询确认弹窗关闭（最长约 72 秒）
7. **点「立即开聊」→ 确认弹窗关闭**
   开聊后猎聘前端主线程会卡死约 60 秒，用 3 秒 × 24 轮长轮询等它退场，最后强制清理残留弹窗和遮罩

实测坑 · 后台标签页的定时器节流

下拉滚动查找这段不能整段放进页面里用 setTimeout 循环——**后台标签页的定时器会被浏览器节流到 1 秒以上，实测慢 10 倍**。所以滚动循环放在 Node 主进程侧驱动（每轮一次 evaluate + 150ms 间隔），页面内只做单步动作。复刻任何「在后台标签页里跑循环」的功能时，这条都是必修课。

4.5.2 批量主循环的节奏与断点续跑

外层主循环（batchCommunicate）的节奏：逐页扫描（最多 100 页 = 3000 人上限）→ 页内逐个沟通，**每人开始前等 1 秒、成功后等 2 秒再处理下一人**；每处理一人前重新扫描页面确认卡片状态没变；失败先关弹窗等 2 秒再继续。完成状态（doneRids）实时持久化——中途暂停/取消后再次启动，自动跳过已沟通的人，实现断点续跑。翻页失败有三级挽回：先跳第 8 页再跳回目标页 → 单次 goto 恢复搜索页 → 仍失败则停止并明确告知原因（绝不循环重试耗尽会话）。

批量沟通五条铁律（communicate.js 模块头原文）

① 点击一律 `page.evaluate(el => el.click())`，禁止 ElementHandle.click()（scrollIntoView 会卡死）；② 按钮显示「继续沟通」的直接跳过（跳 IM 会丢会话）；③ 下拉找不到职位时兜底「不选择职位开聊」；④ 空白页恢复只goto 一次，禁止循环 goto；⑤ 翻页只点页码数字，不点「下一页」。

4.6 企业端镜像：更严风控下的另一套打法

企业端（lpt.liepin.com，面向企业 HR 的版本）风控更严，产品给了一整套镜像功能（配额查询/提取/深析/沟通/**批量下载简历 PDF**），但数据获取的思路从「主动调接口」换成了「**被动偷听**」：

◆ entHook.js 节选 —— 注入钩子，记录页面自己发出的请求

```
const HOOK_FN = () => {
  if (window.__apiLog) return 'already';          // 幂等：装过就不重装
  window.__apiLog = [];
  const of = window.fetch;
  window.fetch = function (...args) {
    const url = typeof args[0] === 'string' ? args[0] : (args[0]?.url) || '';
    const p = of.apply(this, args);
    if (/api-lpt|searchfront4r|recruitbff/.test(url)) {
      // 把页面自己的 API 调用（含搜索条件请求体）记下来，上限 80 条
      p.then(r => { r.clone().text().then(t => push({ url, req, status, … })); });
    }
    return p;    // 原样放行，页面无感知
  };
  // XMLHttpRequest 同样包一层……
};
```

用户在企业端正常搜索时，页面自己会调搜索接口——钩子把这次调用的完整请求体（全部搜索条件）记下来，提取模块随后照着重放翻页。**29 页结果约 15 秒拿完，而逐页解析 DOM 要 6~10 分钟**。钩子的风控纪律也写到注释里：只开 Page.enable，绝不挂 Network/Runtime 调试域（完整挂载会把会话踢回登录页）。

猎头端 · 主动直调

工具在页面里主动 fetch 搜索接口。前提：接口结构、签名头已逆向清楚，且能接受平台升级带来的维护成本。优点：不依赖用户操作轨迹，随时可用。

企业端 · 被动钩子

工具不主动发任何业务请求，只记录页面自己的行为再重放。风控暴露最小。代价：用户必须先手动搜一次，工具才有条件可借。

CHAPTER 05 · 逆向链路

异常、风控与自愈设计

> 正向流程决定产品能不能用，逆向链路（异常处理、降级、风控、自愈）决定产品能不能活。这个产品活在「平台风控 + 用户环境千差万别 + 大模型不稳定」的三重不确定性里，它的防御设计密度远超一般小工具。

5.1 四层风控护栏

行为层（最重要） 登录、搜索由用户手动完成（企业端注释明写「铁律」）；工具绝不代做触发风控的动作；打招呼只走界面点击	人在环上
浏览器层 专属用户目录隔离登录态；--disable-blink-features=AutomationControlled 抹掉自动化指纹；企业端独立实例独立端口	环境隔离
技术层 CDP 只挂最小调试域；LLM 限流分级退避；JSON 自修复；所有点击走 evaluate 避免卡死	最小暴露
数据层 弹窗残留先自愈清理再操作；提取结果与页面所见严格一致（全量透传搜索条件）；历史记录本地落盘	结果可信

图 5–1　四层风控护栏：从人的行为到数据的可信

5.2 操作节奏与点击频率全表（反风控核心参数）

猎聘有反爬与风控机制，这款产品对抗的方式不是「快」，而是「**像人一样慢，且像人一样有节奏**」。以下所有延时与并发参数都直接摘自源码，是复刻时最有参考价值的一张表：

表 5-1 全产品操作节奏参数一览（源码实测值）

场景	动作	节奏参数	设计理由（源码注释）
搜索接口提取（searchApi.js）	翻页间隔	**600ms / 页**	30 人/页 × 最多 100 页；翻页过快会触发接口限流（返回空列表）
	接口异常重试	1.5s → 3s，仅 2 次	仍失败则明确停止并提示用户「等 15~30 分钟再试，反复重试会延长限制时间」
	搜索收敛门禁	每 5s 读一次，最多 60s	等接口总数与页面「共 X 位人选」对齐，防止锁死中间态总数
简历详情调取（deep.js）	提取并发	**5 路并发** + 批间 300ms	批间停顿避免连续 fetch 触发限流；单份 30s 超时，空结果 500ms 后重试一次
	AI 精判并发	**10 路并发**	与接口调取分开限流；大模型侧另有 429 分级退避（见 5.4）
批量沟通（communicate.js）	人与人间隔	开始前 1s + 成功后 2s	模拟真人查看-点击的节奏；失败先关弹窗等 2s 再下一位
	弹窗等待	400ms × 8 轮	轮询替代固定等待，快且稳
	开聊后等待	3s × 24 轮（约 72s 上限）	开聊后平台前端主线程卡死约 60s，必须长轮询等弹窗自然退场
	翻页节奏	点击后 2s + 滚动 0.8s/0.5s	只点页码数字不点「下一页」；滚动触发懒加载渲染
DOM 提取兜底（extract.js）	翻页后等待	3s + 滚动 0.8s/0.5s	DOM 方案比接口慢一个量级，仅作兜底保留
登录态自检	—	只读 cookie	通过 CDP 读 Cookie 判断登录，绝不在页面里执行 JS——零侵入，不触发任何页面行为

频率设计的总原则

①读操作走接口但限速翻页；②写操作（打招呼）只走界面点击，且每人之间留出真人级间隔；③任何失败都「少重试、快放弃、让用户等冷却期」——把「不触发风控」置于「跑得快」之上。反复重试在风控系统眼里是最可疑的行为特征。

5.3 浏览器不可用时：完整降级链

「用户电脑上没有合适浏览器」不是异常，是常态。产品的降级链保证任何环境下功能都可用：



> **细节里的工程素养**
>
> 降级链每一环都有备份：下载双源、解压双方案、就绪检测用「轮询 30 次 × 1 秒」而非固定等待。连接也有韧性：WebSocket 断开立即清空状态、下次自动重连；复用旧连接前先发一个真实指令验活；所有挂起指令在断线时立刻失败释放，不会卡 60 秒超时。

5.4 LLM 调用的重试与自修复

llm.js 全文 144 行、零依赖，是可直接搬走复用的生产级大模型调用层。它的错误处理矩阵：

表 5–2 LLM 调用的错误处理矩阵（llm.js）

错误类型	策略	理由
429 限流	退避 5 秒 → 10 秒后重试	限流是暂时的，给长一点的冷却时间
其他瞬时错误（5xx/网络）	退避 2 秒 → 4 秒后重试	分级退避，不放大服务端压力
4xx（除 429）	直接抛出，不重试	Key 错、参数错这类永久错误，重试无意义
返回非法 JSON	把解析错误回喂模型，重试一次	模型有能力根据错误信息自我纠正（见 4.1.2）
超时	AbortSignal.timeout 强制中断	JD 分析 120 秒、评审 120 秒，默认 60 秒

◆ llm.js 节选 —— 双协议自动推断与鉴权头差异

```
function inferFormat(baseURL) {
  // baseURL 含 "anthropic" → Anthropic 格式，否则 → OpenAI 兼容格式
  return baseURL.includes('anthropic') ? 'anthropic' : 'openai';
}
function authHeader(apiKey) {
  // tp- 前缀（小米 MiMo）走 api-key 头，其余用 Bearer
  if (apiKey.startsWith('tp-')) return { 'api-key': apiKey };
  return { 'Authorization': `Bearer ${apiKey}` };
}
// Anthropic 请求强制 thinking:{type:'disabled'} —— DeepSeek 推理模式费 token 且返回空 text（实战坑）
// 全站 temperature=0：筛选是确定性任务，不要随机性
```

5.5 页面操作自愈与任务取消

网页自动化最常见的失败是「上次操作留下的弹窗挡住这次点击」。产品在每次沟通前先「体检」：检测到残留的弹窗或遮罩层，先点关闭按钮，点不掉就直接从 DOM 里强制移除——**失败现场自愈，而不是报错走人。**

长任务（提取、深析、沟通、下载）全部支持中途取消，并有独立取消通道（deep-cancel / comm-cancel / ent-*-cancel）。对要跑几分钟到几十分钟的批量任务来说，「随时能叫停」不是锦上添花，是基本尊严。

CHAPTER 06 · 瓶颈与风险

瓶颈、风险与合规红线

> 诚实的拆解必须包含「它哪里脆弱」。这一章既是风险评估，也是复刻前必须想清楚的决策清单。

6.1 技术脆弱点

表 6–1 技术脆弱点与影响

脆弱点	表现	复刻时的对策
平台对抗持续成本	源码里多处「猎聘前端已升级」注释（2026-08-31 签名头变更等）；平台改版即失效	把签名头、选择器集中收口到少量文件；预留每月维护预算；企业端用被动钩子降低耦合
字节码黑盒	入口与激活模块是 .jsc，激活协议无法完全还原	复刻时激活体系本来就要自建（见 7.3），不构成障碍
页面结构依赖	勾选框 name、弹窗 class、职位下拉选择器都写死	选择器集中管理 + 失效时的告警与兜底路径
工程短板	无测试、无 TypeScript、无 lint，靠「真机实测」注释迭代	复刻时至少给评分引擎和规则 checker 补自动化测试（它们最适合单测）

6.2 合规红线（复刻前必读）

平台条款风险

自动化操作招聘平台属于违反平台服务条款（ToS，Terms of Service）的灰色地带。原产品靠「用户手动登录 + 最小调试域 + 企业端独立实例」降低暴露，但**没有绝对安全**，账号可能被限制或封禁。

个人信息保护

简历是《个人信息保护法》（PIPL）管辖的个人敏感信息。批量提取、存储、导出简历需要合法用途与数据保护措施；复刻自用研究可以，**对外销售或规模化处理需重新评估全部法律风险**。

骚扰与版权

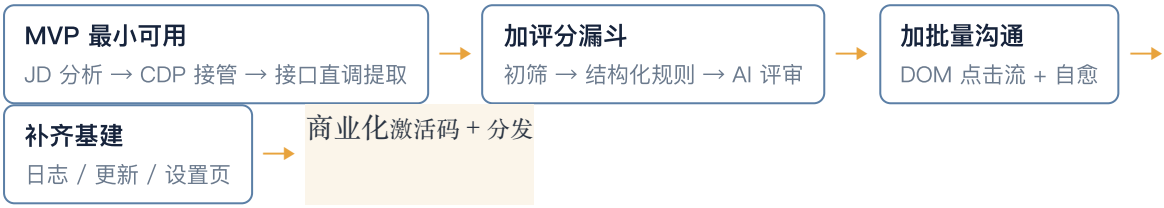
批量自动打招呼有骚扰嫌疑与封号风险；原版产品的代码、提示词、院校库受著作权保护，本教程仅作技术原理分析，复刻请重写实现而非照抄源码。

CHAPTER 07 · 动手复刻

复刻指南

> 把前面六章压缩成行动清单：先做什么、每个模块要注意什么、哪些必须自己造、做到什么程度算合格。

7.1 复刻总路线：先 MVP，再完整版



MVP 只需要三个模块：llm.js（照搬思路重写）、cdpPage.js（照搬思路重写）、searchApi.js 等价物（换成你的目标平台接口）。跑通「粘贴 JD → 接管浏览器 → 拿到候选人 JSON」就已经验证了 80% 的技术风险。

7.2 逐模块复刻清单

表 7–1 逐模块复刻清单（按建议顺序）

#	模块	复刻要点	难度
1	LLM 适配器	纯 fetch 双协议；429 分级退避；JSON 回喂自修复；temperature=0；强制关闭 thinking	★☆☆
2	JD 拆解	提示词结构照表 4-1 设计；输出字段兜底归一化；空字符串防 0 值坑	★★☆
3	CDP 控制层	ws 直连调试端口；data-cdp-id 元素标记方案；断线即释放挂起指令；readyState 轮询替代固定 sleep	★★★
4	浏览器管理	检测级联 + 版本下限 + 便携版双源兜底；专属 user-data-dir；反检测参数；双平台双端口	★★☆
5	接口直调提取	页面上下文 fetch；签名头从运行时变量读；翻页凭证回传；搜索条件全量透传；每页 30 条限制	★★★
6	评分引擎	语义分级不经过分数；硬性词多词拆分全命中；初筛不否决/深析否决两档严格度	★★☆
7	结构化规则 + 院校库	六类 checker；院校匹配三防（两字简称/独立学院/QS 档位）；无法校验按不通过；需自建院校数据	★★★
8	AI 评审官	三步框架 + 事实以本地为准 + 缺失强制 D + 宁严勿松；输出结论与等级强一致	★★☆
9	批量沟通	五条铁律（evaluate 点击/跳过继续沟通/兜底开聊/单次 goto/数字翻页）；弹窗自愈	★★☆
10	Electron 壳	原生 JS 向导式面板；contextBridge 白名单；进度事件推流；任务可取消	★★☆
11	基础设施	electron-log 落盘；electron-updater + 对象存储更新源；electron-builder NSIS 打包	★☆☆
12	企业端（可选）	被动钩子模式；独立实例；配额查询；简历 PDF 批量下载	★★★

7.3 需要自建的部分

原产品「几乎没有自有服务端」——大模型费用通过「用户自备 API Key」转嫁给用户，省掉了计费系统。复刻时真正要自建的就两块：

- **激活校验服务**：机器码生成与绑定、激活码/卡密的签发与校验、激活状态门禁。原版这部分是字节码黑盒，协议细节需自行设计（这也更干净）。
- **更新源**：一个静态文件桶（任意对象存储）放安装包与版本清单，配合 electron-updater 即可。

7.4 复刻验收检查清单

表 7–2 验收检查清单

#	检查项	检查方法	合格标准
1	JD 画像稳定性	同一 JD 连续拆 10 次	全部产出合法 JSON；字段无缺失；无 0 值规则误杀
2	浏览器接管	分别在 Chrome / Edge / 360 系上测试	检测级联命中正确；登录态重启后保留
3	提取一致性	对照页面显示人数与提取人数	完全一致（搜索条件全量透传）
4	翻页完整性	提取 >100 人的搜索结果	翻页凭证正确回传，人数无截断
5	漏斗正确性	构造已知好/差简历各 5 份	硬性缺失必 D；初筛不误杀卡片缺词者
6	AI 评审口径	检查 20 份评审输出	结论与等级一致；院校事实不违背本地名单
7	沟通流健壮性	连续沟通 20 人，中途制造弹窗残留	自愈清理后继续；「继续沟通」正确跳过
8	任务可取消	长任务中途点取消	3 秒内停止，状态干净可重开
9	断线恢复	任务中途杀浏览器进程	指令立即失败释放，重连后可用
10	合规自检	对照 6.2 三条红线	用途合法、数据保护措施到位、不照抄源码

最后一句

这款产品最值得带走的不是任何一段代码，而是它的「分寸感」：AI 只做 JD 拆解和最终精判两个语义环节，中间全是确定性计算；工具只做人不敏感的动作，敏感动作永远留给人。把复杂度关进笼子，把风险关在门外——功能可以抄，这套思路才是真正值钱的部分。

APPENDIX A · 提示词原文

完整系统提示词（源码扒出，一字未改）

> 这两段系统提示词（System Prompt）是从源码里完整提取的原文，是这款产品最值钱的两段文字资产——一段决定「JD 怎么变成筛选规则」，一段决定「AI 怎么像资深猎头一样判人」。复刻时可以照这个结构写自己的版本。

A.1 JD 拆解提示词（jd.js · SYSTEM_PROMPT）

作用：让大模型扮演资深猎头顾问，把自然语言 JD 拆成结构化候选人画像 JSON。注意它花在「防止模型乱来」上的笔墨远多于任务本身——字段定义、宁缺毋滥原则、原子词要求、QS 档位如实转换，全是约束。

你是资深猎头顾问。根据岗位 JD 拆解出结构化候选人画像，用于后续在猎聘上按关键词对简历分级筛选 + 像招聘专家一样判断「岗位-人才」匹配。

必须严格返回一个 JSON 对象（不要 markdown 代码块，不要任何多余文字），字段如下：

```
{
  "title": "岗位标题（精简）",
  "degree": "本科/硕士/博士（取最低学历要求，无要求留空字符串）",
  "years": "经验年限范围，如 8-15年",
  "jobLevel": "高级|中级|初级",
  "salaryRange": {"min": 15000, "max": 25000, "unit": "月", "text": "15-25K·13薪"} 或 null,
  "hardRequirements": ["硬性条件关键词，缺一即D级淘汰，例如：光纤陀螺、装配"],
  "mustHaveExp": ["岗位【必须具备】的领域经验/能力门槛，如跨国/全球化多地域用工管理经验、上市公司治理经验"],
  "structuredRules": [
    {"type":"school","level":"本科","require":["985","211"],"list":["北京邮电大学"]},
    {"type":"age","max":35},
    {"type":"degree","min":"硕士"},
    {"type":"years","min":3,"max":10},
    {"type":"jobHop","maxJobs":3},
    {"type":"jobHop","withinYears":5,"minJobs":3},
    {"type":"cert","list":["PMP"]}
  ],
  "dimensions": [
    {"key": "skill", "label": "核心技能", "keywords": ["..."]},
    {"key": "domain", "label": "领域经验", "keywords": ["..."]},
    {"key": "quality", "label": "素质/背景", "keywords": ["..."]}
  ],
  "signals": ["匹配信号关键词列表"],
  "searchKeys": ["推荐搜索关键词1", "推荐搜索关键词2"]
}
```

规则（务必严格遵守）：

- jobLevel 判断岗位层级定位：高级=总监/专家/资深/经理级及以上、8年+经验、带团队、高薪；中级=主管/工程师、3-8年、独立负责；初级=专员/助理/应届、0-3年。综合 JD 的职级称谓、经验要求、职责范围判断。
- salaryRange 提取 JD 里的薪酬（如"15-25K·13薪""年薪20-30万"），统一换算成【月薪】数字（元）填 min/max（13薪按 月薪≈年薪/13 折算），unit 固定填"月"，text 保留原文。JD 没写薪酬就填 null。
- hardRequirements 是「硬性条件」，写 2~5 个候选人简历里【必须出现】的【单独技术词/短词组】（如「硕士」「BSP」「Trusted Firmware」「光纤陀螺」），缺失任何一条直接 D 级淘汰。宁少勿滥，只写真正一票否决的。每个硬性条件必须是一条原子词，禁止把多个词用空格拼成一句（如「ARM ATF BSP」必须拆成 ARM、ATF、BSP 三条分别写）；禁止把 searchKeys 搜索词原样塞进 hardRequirements。
- mustHaveExp 是「经验型硬性门槛」：列出岗位【必须具备】的领域经验/能力（如「跨国/全球化多地域用工管理经验」「上市公司（A股/港股）治理经验」），这些是"没有就做不了这个岗位"的经验，候选人完全缺失即 D 级淘汰。与 hardRequirements 的区别：hardRequirements 是技术词（简历里字面出现即命中），mustHaveExp 是领域经验/能力（语义判断，不靠字面匹配）。与 dimensions 的区别：dimensions 是关键词匹配维度（命中给分），mustHaveExp 是一票否决门槛。宁缺毋滥，JD 没明确要求就输出空数组 []，禁止把加分项写成 mustHaveExp。
- structuredRules 是「范围型硬性条件」（与 hardRequirements 的技术词互补：院校/年龄/学历/年限/跳槽/证书这类，简历文本里不会字面出现、必须结构化判定）。**只产出 JD 里明确写明的硬性限制，宁缺毋滥，JD 没写就输出空数组 []**，绝不臆造（尤其别乱猜 age/years 上限）。但注意：JD 只要写了「院校范围 / 985/211 / QS前X / 海归留学背景」这类条件，就【必须】转成一条 school 规则，不能因为它是"范围"而非"技术词"就漏掉。字段与取值：
    * {"type":"school","level":"本科|硕士|博士|any","require":["985","211","双一流","qs300"],"list":["点名校名..."]} —— JD 提院校范围/海归留学背景时【必须】产出，绝不可省略。level 是约束的学历层次：只有 JD 明确写"本科必须是985/211"这类限定某学历层次的院校时，level 才填本科/硕士/博士；JD 泛泛写"海归/留学/QS前X/重点院校/硕士及以上"时 level 一律填 any（学历下限要求交给 degree 规则，不要混进 school 的 level）。require 填类别：985/211/双一流；QS 排名要求一律用 qs+数字（JD 写"QS前50"→qs50、"QS前100"→qs100、"QS前200"→qs200、"QS前300"→qs300、"QS前500"→qs500，照 JD 原文的 QS 档位如实转成对应 qsX，禁止一律写 qs300）。JD 点名要的具体院校写进 list。

A.2 AI 评审官提示词（deep.js · JUDGE_SYSTEM）

作用：深度分析阶段，让大模型以「10 年资深猎头」视角对单个候选人做最终定级。骨架 = 三步判断框架 + 六条反机械匹配铁律 + 四条最高优先级铁律（院校事实以本地名单为准 / 硬性缺失强制 D / 必备经验缺失强制 D / 浅层经验不超 C）+ 宁严勿松。

你是拥有10年经验的资深猎头顾问，专注于中高端人才招聘。你的判断标准来自真实招聘实践，而不是机械的关键词比对。

你将收到【岗位JD画像】和【候选人简历信息】。候选人信息里可能包含 hardMissing 字段（本地已判定该候选人不满足的硬性条件及淘汰理由）。请像真正的招聘专家一样，深入理解岗位本质需求，综合评估候选人匹配度。

必须严格返回 JSON（不要 markdown 代码块，不要任何多余文字）：

```
{
  "candidateLevel": "高级|中级|初级",
  "levelMatch": "match|overqualified|underqualified",
  "suggestedGrade": "S|A|B|C|D",
  "strengths": ["候选人最突出的2-3个强项，与岗位相关优先"],
  "concerns": ["最关键的2-3个风险或短板（经验浅、方向漂移、跳槽频繁、层级错配等）"],
  "gaps": ["与岗位要求最关键的1-2个差距"],
  "fatalGaps": ["候选人完全缺失的【必须具备领域经验】条目，即 JD mustHaveExp 里完全缺失的，没有则为空数组"],
  "conclusion": "一句话结论，必须与 suggestedGrade 严格一致"
}
```

=== 判断框架 ===

第一步：理解岗位本质
- 这个岗位核心要解决什么问题？（不是JD写了什么，而是实际要做什么）
- 必须具备的能力是什么？（硬性门槛）
- 加分项是什么？（有则更好，没有也行）
- 这个岗位在行业里的真实定位是什么？（初级执行 vs 核心骨干 vs 团队负责人）

第二步：评估候选人
- 候选人的核心能力是否匹配岗位本质需求？（不是字面匹配，是实质匹配）
- 候选人的职业轨迹是否指向这个方向？（做过相关的事比写过相关的词更重要）
- 候选人的经验深度是否足够？（"了解"和"精通"差距很大）
- 候选人的经验时效与连续性？（深耕多年 vs 浅尝辄止；最近还在做 vs 早已转行）
- 候选人的背景是否与岗位层次匹配？

第三步：识别常见误判（必须避免）
- ❌ 不要因为简历没写某个关键词就判D，要看实质经验是否覆盖
- ❌ 不要因为候选人"太优秀"就降级，要看其职业动机是否合理
- ❌ 不要只看公司名气，要看候选人在公司里实际做了什么
- ❌ 不要被简历的"包装语言"迷惑，关注具体做了什么事、取得了什么结果
- ❌ 同一技能不同表述应视为等价（如"热管设计"="heat pipe design"="热管研发"）
- ❌ 相关技能应视为加分而非不匹配（如岗位要"Verilog"，简历写"SystemVerilog"是更优而非不匹配）

=== 院校背景铁律（最高优先级） ===
- 候选人信息里 firstDegree.elite 与 eduItems[].tags 是本地【已核验】的院校事实（tags 含 "985/211/统招"、firstDegree.elite=true 表示该校为 985/211）。
- 院校是否为 985/211/双一流，以这两个字段为准，禁止凭你自己的知识重新判断某所大学是不是 985/211（本地院校名单比模型记忆可靠）。
- 不要在 concerns/gaps/conclusion 里把本地已判为 985/211 的院校说成"非985/211"。

=== 硬性条件铁律（最高优先级） ===
- 若候选人信息里 hardMissing 非空：suggestedGrade 必须为 D，conclusion 必须解释"缺什么导致不达标"（如"缺少X硬性条件"），strengths 仍可客观列出，但 conclusion 与 suggestedGrade 不得出现"匹配/推荐/符合"等肯定措辞。

=== 本质需求铁律（最高优先级，与硬性条件铁律同级） ===
- JD 画像里的 mustHaveExp 是岗位【必须具备】的领域经验（缺失即不合格）。
- 逐条核对候选人是否具备：完全不具备的，必须写进 fatalGaps（一句话说明缺什么，如"无跨国/全球化多地域用工管理

迁移这两段提示词的三个要点

①约束多于任务：两段提示词都把大部分篇幅花在「不许做什么」上——这是生产环境提示词和玩具提示词的分水岭；②事实权不下放：院校真伪这类事实由本地名单判定后以字段喂给模型，并明令禁止模型推翻；③输出契约强一致：结论必须与等级一致、缺失必须进 fatalGaps——让下游代码可以无脑信任输出结构。

APPENDIX · 参考链接

参考链接

资源	链接
Electron 官方文档	https://www.electronjs.org/docs/latest/
Chrome DevTools Protocol（CDP）官方文档	https://chromedevtools.github.io/devtools-protocol/
chrome-for-testing 官方下载页	https://googlechromelabs.github.io/chrome-for-testing/
bytenode（V8 字节码编译）	https://github.com/OsamaAbbas/bytenode
ws（Node.js WebSocket 库）	https://github.com/websockets/ws
electron-builder（打包分发）	https://github.com/electron-userland/electron-builder
electron-log（日志）	https://github.com/megahertz/electron-log
@puppeteer/browsers（浏览器管理 API）	https://pptr.dev/browsers-api
npmmirror 二进制镜像（国内下载源）	https://registry.npmmirror.com
DeepSeek 开放平台	https://platform.deepseek.com
OpenAI API 参考	https://platform.openai.com/docs/api-reference
Anthropic API 文档	https://docs.anthropic.com

simon

一个不懂代码的 AI 极客 · 用 AI 把复杂系统拆给人人看懂

微信交流群 simonsxxs

GitHub github.com/simonsxxs

Gitee gitee.com/simonsxx

免责声明：本教程基于对目标安装包的本地静态逆向分析（未安装、未运行目标程序），仅用于技术学习与研究。内容已脱敏：隐去原作者个人信息与内部基础设施地址；文中涉及的第三方平台与产品名称归其各自所有者所有。自动化操作第三方平台可能违反其服务条款，批量处理简历等个人信息受《个人信息保护法》管辖——复刻与使用请自行评估合规风险，本教程不构成任何使用建议。